FuelPrint team plan

KIET InnoTech26 · Category 2, AI Solutions for Automation · for all five of us · 16 Oct 2026

What we are building

FuelPrint helps a pollution investigator decide where to measure next, compare the factories that could explain a pollution spike, and keep an honest record of what the evidence does and does not show.

It runs as three web apps on one shared server:

- **Operations** is the control room. Sensor boxes report gas readings. When a box sees a sustained sulphur rise, the system opens an investigation, ranks which factories could explain it, and proposes the next measurement. A supervisor approves it.
- **Field** is the 3D world. A drone flies to the approved spot (a visitor can fly it from their phone), takes a sample at the real position of its inlet, and the sample is checked channel by channel.
- **Cases** is the evidence side. It keeps the screening record, a separate inspector specimen with seals and handovers, a lab result, a written finding, and a sealed evidence pack that anyone can verify.

Everything is a simulation of a fictional industrial estate laid on the real Kundli map. We never name a real factory, never claim to detect a fuel, and never call anything legal proof.

This is the same FuelPrint that won IDEAVERSE. The early pitch said "find the factory burning coal at night". Our research showed that sensors cannot prove which fuel a factory burns, and that a box can miss a plume completely. So the product is now an investigation tool that is honest about what it can't tell, and that honesty is what we pitch.

The five of us

Person	Group	Main job
Aditya Singh	Field	Owns the 3D world, drone, phone joystick. Runs the AI tools that build the shared core and the science engine. Merges everyone's work.
Aditya Singhal	Operations	Builds the Operations backend and most Operations screens with Codex
Abhivansh	Operations	Builds the sensor health panel and the hardware page. Main job: study the whole idea, the hardware and the 92 judge questions. He is our pitcher.
Antara	Cases	Builds the Cases backend with Codex: cases, specimens, custody, lab maths, findings, the evidence vault
Aditya	Cases	Builds all Cases screens with Codex

To avoid mix-ups: "Aditya" alone in these docs always means the Aditya in Cases. The other two are always written with their surnames.

How three groups work at the same time

Normally the screens wait for the backend and the backend waits for the science. We avoid that in four ways.

1. One agreement. The file `contracts/AGREEMENT.md` in the repo fixes the exact shape of every message, command and record. Everyone builds against it. Only Aditya Singh changes it. If you think something is missing, ask in the group; don't change it yourself.

2. A fake server first. On 17 Oct the shared server gets a "replay" mode. It plays recorded test data through the same addresses the real server uses. Your screens work against it from day one. When the real parts arrive you change nothing in your code.

3. Stubs for the science. The science engine (ranking, planner, readings) is built separately by Snowflake Cortex. Until it is ready, the server uses stub versions that return fixed answers. Operations and Field code calls the same Java interfaces either way.

4. Own folders. Each person edits only their own folders (list below). Two people never edit the same file, so merges stay easy.

On **20 Oct** we join everything: fake data is switched off, real science is switched on, and we fix mismatches together.

Who builds what

All coding is done through AI. You give Codex the prompts from your guide, check what it made, run the tests, and push.

Part	Built by	Runs on
Agreement, shared look (colours, buttons), reviews and merges	Claude	Aditya Singh's Mac
Shared core: server, database, live updates, clock, logins, fake server, start scripts	Codex	Aditya Singh's Mac
Science engine: pollution simulation, sensor behaviour, alarm rule, factory ranking, next-measurement planner, test scenarios	Snowflake Cortex	Aditya Singh's Mac
Field: 3D world, drone, phone joystick, capture and checks	Claude (visuals) and Codex (code)	Aditya Singh's Mac
Operations backend and screens	Aditya Singhal with Codex	His laptop
Sensor health panel and hardware page	Abhivansh with Codex	His laptop
Cases backend	Antara with Codex	Her laptop
Cases screens	Aditya with Codex	His laptop
Backup worker when Codex or Cortex hit limits	Kiro	Aditya Singh's Mac

Languages and why

Layer	Choice	Why
Screens	React 18 + TypeScript, built with Vite	Fast to build, the AI tools know it well, type errors show up before the demo
3D	Three.js used directly	Full control over the scene and frame rate. Only Field uses it.
Charts	Recharts	Simple React charts for live readings
Backend	Java with Spring Boot 3.5, Maven wrapper	One language for the whole backend, easy for all of us to read
Database	SQLite through plain SQL (Spring JdbcTemplate, Flyway for tables)	Real SQL in one file. Nothing to install on stall laptops, and a backup is just a copy of that file.
Live updates	WebSocket (Spring)	The server pushes 10 updates a second to every screen and phone

Layer	Choice	Why
Maths	Apache Commons Math	Matrices and least-squares fitting for the science engine
Evidence	SHA-256 from Java's standard library; Ed25519 signing in the finale	Built into Java, no extra service

Java version: any JDK 17 or newer works (we compile for 17). Install Temurin 21 if you have nothing. Node 22 LTS for the screens.

The repo

One private GitHub repo: **github.com/adityasinghin01-hash/fuelprint-innotech**. Send Aditya Singh your GitHub username so he can add you.

```
contracts/          the agreement + reference numbers
Aditya Singh only
server/             one Spring Boot app (Java)
  .../fuelprint/core      server, database, live, clock
Codex (Aditya Singh)
  .../fuelprint/api       interfaces between parts
Codex (Aditya Singh)
  .../fuelprint/science   simulation, ranking, planner
Snowflake Cortex
  .../fuelprint/ops       Operations backend
Aditya Singhal
  .../fuelprint/cases     Cases backend
Antara
  .../fuelprint/field     drone, leases, captures
Aditya Singh
apps/shared/        shared look, types, live connection
Claude
apps/operations/    Operations screens
Aditya Singhal, Abhivansh
apps/cases/         Cases screens
Aditya
apps/field/         3D world and phone page
Aditya Singh
data/context/       Kundli map crop and wind (real data)
read only
docs/plan/          these documents
tools/              start scripts
Codex (Aditya Singh)
```

How to work with git

1. Each person works on their own branch: `ops/singhal`, `ops/abhivansh`, `cases/antara`, `cases/aditya`.
2. Commit small pieces often, with a clear message ("ops: approve mission screen").
3. Push and open a pull request into `main` at least once a day, by 9 PM.
4. Claude reviews every pull request on Aditya Singh's Mac. It merges if the tests pass and you only touched your own folders. Otherwise it comments with what to fix.
5. Every morning run `git pull origin main` and merge `main` into your branch before you start.

Never push straight to `main`. Never commit passwords, keys or `server/config/stations.json`.

The agreement in short

The full file is `contracts/AGREEMENT.md`. These are the parts everyone must know.

- **The server decides everything.** Your screen shows what the server sends and asks it to do things. It never calculates a reading, a score or an outcome itself.
- **A command gets a receipt.** To change anything, send `POST /api/commands` with a fresh `command_id`. The reply says APPLIED or REJECTED with a reason. If you retry, use the same `command_id`; the server will not do it twice.
- **Revisions stop overwrites.** Each command carries the `expected_revision` you last saw. If someone changed the record in between, you get `STALE_REVISION`. Reload and show the new state.
- **Live data comes from one socket.** `/ws/live` sends a `tick` 10 times a second (drone, readings, wind, clock) and an `event` whenever a record is saved. After a reconnect, ask for events after your last `cursor`.
- **Stale means read-only.** If nothing arrives for 3 seconds, show "stale" and disable buttons.
- **Unknown is null, never zero.**
- **Three outcomes are normal.** Supported, rejected and inconclusive all get a proper screen, not an error box.
- **Scores are not chances.** The ranking shows a fit number. Never write it as "87% guilty".
- **Hidden truth stays hidden.** No screen, API or AI text may know which factory is really emitting.

What we show at the department round

Five laptops and visitors' phones, all on one Wi-Fi.

Laptop	App	Who stands there
1	Operations: health, investigation, approval	Aditya Singhal
2	Field: 3D world and drone (also the host running the server)	Aditya Singh
3	Cases: lab check and lab result	Antara
4	Cases: inspector, custody, evidence vault	Aditya
5	Operations: board decision and public notice	Abhivansh (also pitches)

The main story (about 6 minutes)

1. **Laptop 1.** The run starts on the fictional estate at night. Four sensor boxes report SO2, CO2 and particles every second. Box N2's SO2 climbs and stays up for 15 seconds, so an alarm opens an investigation.
2. **Laptop 1.** The ranking shows which factories could explain it. Two still fit (S01 and S02). The screen also shows what it can't tell and which readings it ignored.
3. **Laptop 1.** The planner proposes the next measurement: fly the drone to a point downwind of both candidates at 35 m height and collect 30 seconds of valid air. It shows a cheaper option too. The supervisor approves.
4. **Laptop 2 and a phone.** A visitor scans the QR code, gets a 6-letter code, and flies the drone. The readings depend on where the drone's inlet really is. At the target it waits, purges, settles and collects.
5. **Laptop 2.** The capture is checked channel by channel. SO2 valid, CO2 valid.
6. **Laptop 1.** The ranking updates. Now only S02 fits, inside this synthetic model.
7. **Laptops 3 and 4.** Cases receives the screening record. The inspector records a separate specimen with a seal, hands it over, the lab enters a result, and a finding is written with its limits. Legal suitability always says "not assessed".
8. **Laptop 4.** The case is sealed. "Verify" checks every byte. "Tamper" changes one byte in a copy, and verify catches it.
9. **Laptop 5.** The board records a decision and previews an area-level public notice that names no factory.

Two more stories that prove we are honest

- **Blocked inlet.** Same start, but the SO2 tube blocks during capture. The number on screen still looks normal, but the flow

is zero. SO2 is rejected and CO2 is still usable. The case ends "rejected for use".

- **Plume above the boxes.** A hot plume from a 45 m stack passes over every 3 m box. No alarm fires. The analyst schedules a survey, the drone result fits two explanations, and the case ends "inconclusive". A missed plume never becomes a "clean" verdict.

Judges can fly the drone, pick a scenario, block the inlet, and tamper with the evidence.

Dates

Date	What happens
Fri 16 Oct	Docs, repo and agreement done. Send your GitHub username. Set up your laptop (last section).
Sat 17 Oct	Codex builds the shared core and fake server. You start screens against the example data in `apps/shared/fixtures`. Cortex starts the science engine.
Sun 18 to Mon 19 Oct	Everyone builds their steps. Pull requests every night.
Tue 20 Oct	Join day. Real science on, fake data off. Fix mismatches.
Wed 21 Oct	Full rehearsal on five laptops and three phones. Freeze the department version at 10 PM.
22 to 24 Oct	Department round (exact slot not fixed yet). Only bug fixes on the frozen version.
25 to 29 Oct	Finale work (list below), rehearsals, backup laptop drill.
Fri 30 Oct	Grand finale

What the finale adds

- More scenarios: wind turning mid-mission, two factories at once, a sensor fault, an unregistered source, a weak plume with background drift, combined errors.
- Operations: sensor placement and coverage view, teaching controls to change wind and stack height live.
- Field: instrument cutaway showing the pumps and gas paths, a lab workbench in 3D, the ground van, more polish.
- Cases: signed evidence (Ed25519), an export file, and a verifier that runs on a different laptop.
- Reliability: restart recovery, backup host, 60-minute rehearsal.
- Optional: the ₹2,000 bench sensor box, shown only as a relative dust reading, never as gas or fuel detection.

Things nobody says on stage

- "It detects coal." It can't. It shows a sulphur-bearing clue at most.
- "This factory is guilty." We show which registered sources fit the model, plus unknown sources.
- "87% probability." Our scores are fit numbers, not probabilities.
- "This is legal evidence." The inspector flow is a demo of the workflow. Legal suitability is always "not assessed".
- "No alarm, so the area is clean." A box can miss a plume completely.
- "Our hardware is tested." No physical test has been done yet. The hardware plan is a costed plan.
- "We have customers." We don't yet.

Set up your laptop today

1. Install **Git**. Check with `git --version`.
2. Install a **JDK 17 or newer** (Temurin 21 from adoptium.net if you have none). Check with `java -version`.
3. Install **Node 22 LTS** from nodejs.org. Check with `node -v`.
4. Make sure **Codex** runs: open a terminal and type `codex`. Sign in if asked.
5. Send Aditya Singh your **GitHub username**. Accept the invite email.
6. Clone the repo: `git clone https://github.com/adityasinghin01-hash/fuelprint-innotech.git`
7. Create your branch: `git checkout -b ops/singhal` (use your own branch name from the list above).
8. Read your group guide, then `contracts/AGREEMENT.md`.

From Saturday evening these commands work (Codex builds them first):

Command	What it does
`tools/start-fake.sh`	Starts the server with recorded test data on port 8080
`tools/start-real.sh`	Starts the real server
`tools/dev-app.sh operations`	Runs one app with live reload on port 5173 (also `cases`, `field`)
`tools/test-all.sh`	Runs every Java and screen test
`tools/stall.sh`	Builds everything and starts the stall version on the host

On Windows use the `.ps1` versions of the same scripts.

When stuck: first paste the exact error into Codex and ask it to fix it. If that fails twice, message the group with the error text, what you tried, and the file name.